

NYC Food Safety Dashboard Presentation Study Guide

A structured review guide for your 6-minute final presentation, dashboard walkthrough, and Q&A.

Driving question: How do food safety outcomes vary across restaurants, boroughs, cuisines, and inspection patterns in New York City, and how can those patterns help users make safer dining decisions?

1. The 6-Minute Presentation Flow

Time	What To Cover
0:00 - 0:45	Introduce the project, the driving question, and the audience. Say that the project uses NYC restaurant inspection data and ends as a Dash dashboard.
0:45 - 1:30	Explain the data work: standardizing columns, converting dates and scores, handling missing values, removing duplicates, and creating a latest-inspection restaurant snapshot.
1:30 - 2:15	Show the KPI cards and the borough comparison chart. Explain that these provide the citywide overview and borough-level contrast.
2:15 - 3:00	Show the trend chart and cuisine chart. Explain why time and restaurant type add analytical depth beyond one static score.
3:00 - 4:00	Show the violation chart and the map. Explain that the violation summary simplifies long inspection language and the map turns the analysis into restaurant-level exploration.
4:00 - 5:00	Explain the app architecture: notebook for analysis, data_utils.py for cleaning, app.py for layout and callbacks, and CSS for design.
5:00 - 6:00	Close with limitations, future improvements, and why the dashboard is useful for both an academic and practical audience.

Opening line: My project is an interactive dashboard on New York City food safety. I started with restaurant inspection data, cleaned and structured it, engineered features such as risk level and violation categories, and then converted the work into a Dash application for interactive presentation.

2. Dashboard Components and What To Say

KPI cards

These summarize the current filtered view: number of restaurants, A grade share, average score, and critical violation share. They give the audience a fast overview before the detailed visuals.

Borough comparison

This compares boroughs using average score and critical violation rate. It helps answer whether safety patterns vary spatially across major city regions.

Trend over time

This shows how the average inspection score changes over time. It prevents the project from being only a static snapshot and adds a historical perspective.

Cuisine risk landscape

This compares restaurant types using average score, critical violation rate, and restaurant count. It helps show which cuisine groups have relatively different recent safety profiles.

Violation category summary

This groups long violation descriptions into broader categories so the audience can see the most common types of inspection issues without getting lost in raw text.

Interactive map

This is the most practical view. It lets a user search restaurants and explore location, grade, score, risk level, and violation summary together.

Inspection detail card

When a restaurant is clicked on the map, this panel reveals the full inspection comment and address. It demonstrates summary-to-detail interaction, which is a core interactive data principle.

Safer picks table

This translates analysis into action by surfacing restaurants with stronger recent safety indicators.

3. The Technical Story of the Project

- 1 Stage 1: Data wrangling. I standardized the raw NYC inspection dataset, fixed column names, converted data types, handled missing values, removed duplicates, and filtered unusable inspection records.
- 2 Stage 2: Feature engineering. I created risk levels from inspection scores, time fields such as year and year-month, simplified violation categories, and a latest-inspection restaurant snapshot.
- 3 Stage 3: Dashboard development. I built the dashboard in Dash, added filters and callbacks, styled the interface with CSS, and made the final product locally runnable on localhost.

Why there are multiple files: The project is split by responsibility. The notebook documents analysis and decisions. `data_utils.py` handles cleaning and feature engineering. `app.py` contains the dashboard layout and callbacks. `assets/styles.css` controls the visual design.

4. Questions You Should Be Ready To Answer

Q: Why did you use Dash instead of only a notebook?

A: Because a notebook is good for analysis, but Dash is better for presenting an interactive final product where the audience can explore filters and map interactions.

Q: What kind of data wrangling did you do?

A: I standardized columns, converted dates and numeric fields, cleaned text values, handled invalid placeholders such as borough value 0, removed duplicates, and built both inspection-level and restaurant-level datasets.

Q: What is feature engineering in this project?

A: Feature engineering includes risk level, year and year-month fields, simplified violation categories, restaurant display labels, and the latest-inspection restaurant snapshot.

Q: Why does the dashboard use the latest inspection for some charts?

A: Because restaurant-level comparisons should reflect the most current state of each restaurant rather than overcounting businesses with many inspection rows.

Q: What does records mean in the violation chart?

A: It means inspection rows, not unique restaurants. A single restaurant can contribute more than one inspection record over time.

Q: Why is localhost useful for a class project?

A: It is reliable, fast, and avoids free-tier hosting problems such as sleeping servers, cold starts, or internet dependence during a live presentation.

Q: What are the limitations of the project?

A: The data is observational, some raw fields are missing or invalid, the violation grouping is a simplified categorization, and the map can only show rows with valid coordinates.

Q: What would you improve next?

A: I would add deeper borough-over-time analysis, richer text analysis of violations, and possibly deploy the dashboard publicly after local testing is complete.

5. Final Submission and Demo Checklist

- Make sure the final notebook is uploaded to GitHub.
- Make sure the repository includes the data file used by the project. In this version, the dataset is packaged as `data/nyc_inspection_data.csv.gz`.
- Make sure the repository includes `app.py`, `data_utils.py`, `requirements.txt`, and `assets/styles.css`.
- Before class, test the dashboard locally by running `python3 app.py` and opening `http://127.0.0.1:8050`.
- Practice the six-minute flow and decide in advance which dashboard interactions you will demonstrate live.

- Keep your conclusion short: summarize the driving question, method, interactive design, and main insight.

Closing line: Overall, this project shows how messy public inspection data can be cleaned, structured, analyzed, and turned into an interactive dashboard that supports both data interpretation and practical decision-making.